

机器学习基础作业 8

宋思逸

2400010816

2026 年 5 月 28 日

1 简述如何基于矩阵奇异值分解求规范化的样本矩阵的前 k 主成分矩阵

设有 n 个样本，每个样本有 p 个特征。考虑样本矩阵 $\mathbf{X} \in \mathbb{R}^{n \times p}$ ，其中每一行是一个样本，每一列是一个变量/特征。对每一列进行规范化（常用标准化，使得每个特征大致满足均值为 0，标准差为 1）。

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j} \quad (1)$$

得到 $\mathbf{Z} \in \mathbb{R}^{n \times p}$

主成分分析（PCA）希望找到一些新的彼此正交的方向，使得数据在这些方向上的变化最大，并且这些方向彼此正交。

对规范化样本矩阵 $\mathbf{Z}$ 做奇异值分解

$$\mathbf{Z} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (2)$$

其中 $\mathbf{\Sigma}$ 为奇异值矩阵，对角线上是奇异值，通常按照从大到小排列 $\sigma_1 \geq \sigma_2 \geq \dots \leq \sigma_r \geq 0$ ，其中 $r = \text{rank}\mathbf{Z}$ ，奇异值越大表示对应方向上的信息量越大。

规范化矩阵 $\mathbf{Z}$ 的协方差矩阵为

$$\mathbf{S} = \frac{1}{n-1} \mathbf{Z}^T \mathbf{Z} \quad (3)$$

代入奇异值分解

$$\mathbf{Z} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (4)$$

则

$$\mathbf{Z}^T \mathbf{Z} = \mathbf{V}\mathbf{\Sigma}^T \mathbf{U}^T \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T \quad (5)$$

注意到 $\mathbf{U}^T \mathbf{U} = \mathbf{I}$ 从而

$$\mathbf{Z}^T \mathbf{Z} = \mathbf{V}\mathbf{\Sigma}^2 \mathbf{V}^T \quad (6)$$

于是

$$\mathbf{S} = \frac{1}{n-1} \mathbf{V}\mathbf{\Sigma}^2 \mathbf{V}^T \quad (7)$$

我们得到协方差矩阵 $\mathbf{S}$ 的特征向量就是 $\mathbf{V}$ 的列向量，特征值为

$$\lambda_i = \frac{\sigma_i^2}{n-1} \quad (8)$$

从 $\mathbf{V}$ 中取前 k 列

$$\mathbf{V}_k = [v_1, v_2, \dots, v_k] \quad (9)$$

其中 v_t 为第 t 主成分方向，矩阵 $\mathbf{V}_k \in \mathbb{R}^{p \times k}$ 为前 k 个主成分方向矩阵/载荷矩阵。计算

$$\mathbf{Y}_k = \mathbf{Z}\mathbf{V}_k \quad (10)$$

则 $\mathbf{Y}_k$ 就是前 k 主成分得分矩阵

2 求协方差矩阵最大特征值的计算方法

这种方法叫做幂法。神秘的是，我还真就在计算方法课上见到过这个方法。

基本思想：任取初始向量，使用 S 乘它，经过多次迭代之后，这个向量会越来越接近最大特征值对应的特征向量。

我们假设矩阵 S 的特征值降序排列：

$$\lambda_1 > \lambda_2 \geq \lambda_3 \geq \cdots \quad (11)$$

其中 λ_1 为最大特征值。

任意一个初始向量 x_0 通常可以表示为多个特征向量的线性组合

$$x_0 = c_1 v_1 + c_2 v_2 + \cdots + c_p v_p \quad (12)$$

其中 v_i 为 λ_i 对应的特征向量

于是

$$S^m x_0 = c_1 \lambda_1^m v_1 + c_2 \lambda_2^m v_2 + \cdots + c_p \lambda_p^m v_p \quad (13)$$

当 m 足够大后，最大特征值的方向会占据主导。

下面给出幂法的具体步骤：

已知协方差矩阵 $S \in \mathbb{R}^{p \times p}$ ，要求最大特征值 λ_1

(1) 任取非零向量 $x^{(0)}$ 并单位化

$$x^{(0)} = \frac{x^{(0)}}{\|x^{(0)}\|} \quad (14)$$

(2) 第 t 次迭代时，计算

$$y^{(t+1)} = Sx^{(t)} \quad (15)$$

随后单位化

$$x^{(t+1)} = \frac{y^{(t+1)}}{\|y^{(t+1)}\|} \quad (16)$$

(3) 当 $x^{(t)}$ 比较稳定（例如 $\|x^{(t+1)} - x^{(t)}\| < \varepsilon$ 的时候），可以用 Rayleigh 商估计最大特征值

$$\lambda_1 \approx \frac{(x^{(t)})^T S x^{(t)}}{(x^{(t)})^T x^{(t)}} \quad (17)$$

注意停止条件也可以是 $|\lambda^{(t+1)} - \lambda^{(t)}| < \varepsilon$

3 自选一公开图像数据集，利用主成分分析方法进行降维实验，观察不同维数下的图像

"""PCA 图像降维实验 —— Olivetti (ORL) 人脸数据集"""

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.datasets import fetch_olivetti_faces

# ----- 1. 载入数据 -----
faces = fetch_olivetti_faces(shuffle=True, random_state=42)
X = faces.data # (400, 4096), 每行是一张拉直的人脸, 像素已归一到
n_samples, n_features = X.shape
img_h, img_w = 64, 64 # 每张图 64x64

# ----- 2. 数据中心化 -----
mean_face = X.mean(axis=0) # 平均脸 (4096,)
X_centered = X - mean_face # 去均值

# ----- 3. 基于 SVD 求主成分 -----
#  $X_{centered} = U @ \text{diag}(S) @ V^T$ ;  $V^T$  每一行是一个主方向 (特征脸)
U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
components = Vt # (400, 4096)
eigenvalues = (S ** 2) / (n_samples - 1) # 协方差矩阵特征值
evr = eigenvalues / eigenvalues.sum() # 各主成分方差占比
cum_evr = np.cumsum(evr) # 累计方差占比

# ----- 4. 投影 / 重构函数 -----
def reconstruct(xc, comps, mean, k):
    Wk = comps[:k] # 取前 k 个主成分 (k, d)
    z = xc @ Wk.T # 投影到 k 维 (n, k)
    return z @ Wk + mean # 重构回 d 维 (n, d)

# ----- 5. 同一张脸在不同维数下重构 -----
k_list = [1, 5, 10, 25, 50, 100, 200, n_samples]
idx = 0 # 选第 0 张脸观察

fig, axes = plt.subplots(1, len(k_list) + 1, figsize=(16, 3))
axes[0].imshow(X[idx].reshape(img_h, img_w), cmap='gray')
axes[0].set_title('Original'); axes[0].axis('off')
for ax, k in zip(axes[1:], k_list):
    rec = reconstruct(X_centered[idx:idx + 1], components, mean_face, k)
    ax.imshow(rec.reshape(img_h, img_w), cmap='gray')

```

```

    ax.set_title(f'k={k}\n{cum_evr[k - 1] * 100:.1f}%'); ax.axis('off')
plt.suptitle('PCA reconstruction at different dimensions')
plt.tight_layout(); plt.savefig('py_reconstruction.png', dpi=150)

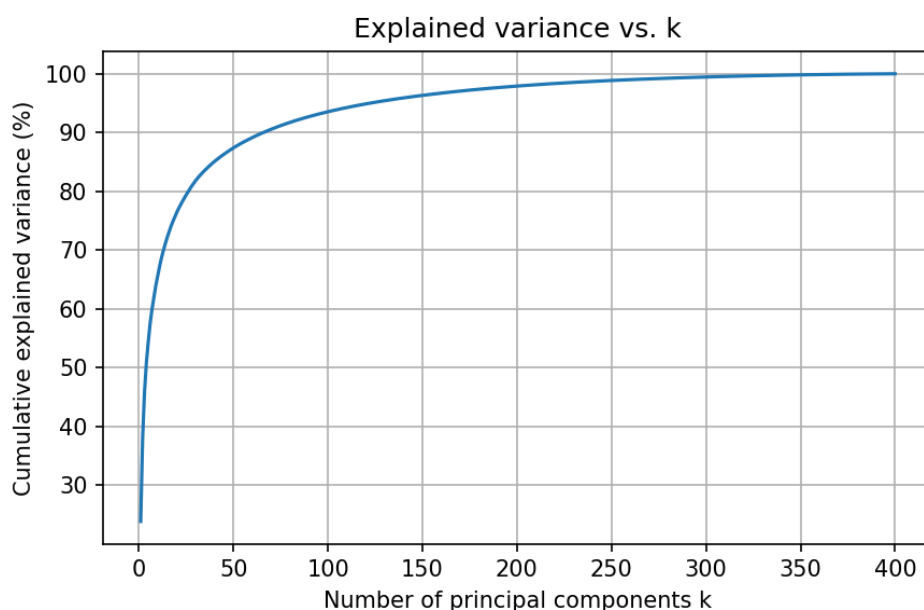
# ----- 6. 平均脸与前 11 个特征脸 -----
fig, axes = plt.subplots(2, 6, figsize=(12, 4.5)); axes = axes.ravel()
axes[0].imshow(mean_face.reshape(img_h, img_w), cmap='gray')
axes[0].set_title('Mean face'); axes[0].axis('off')
for i in range(1, 12):
    axes[i].imshow(components[i - 1].reshape(img_h, img_w), cmap='gray')
    axes[i].set_title(f'PC {i}'); axes[i].axis('off')
plt.tight_layout(); plt.savefig('py_eigenfaces.png', dpi=150)

# ----- 7. 累计方差贡献率曲线 -----
plt.figure(figsize=(6, 4))
plt.plot(np.arange(1, len(cum_evr) + 1), cum_evr * 100)
plt.xlabel('Number of principal components k')
plt.ylabel('Cumulative explained variance (%)')
plt.title('Explained variance vs. k'); plt.grid(True)
plt.tight_layout(); plt.savefig('py_explained_variance.png', dpi=150)

plt.show()

```

我们看一个趋势图



观察图像，横坐标为主成分数目，纵坐标为还原度。可以看出一开始斜率非常大，证明前几个主成分包含了很多信息，而后面的主成分包含的信息并不多，且只要 400 个就能够基本完全还原信息了。

注：数据集选择：Olivetti / ORL 人脸库